```
# on your staging environment
```

The default queue name prefix delimiter is '_'. This can be changed by setting `config.active_job.queue_name_delimiter` in `application.rb`:

```
# config/application.rb
module YourApp
  class Application < Rails::Application
    config.active_job.queue_name_prefix = Rails.env
    config.active_job.queue_name_delimiter = '.'
  end
end

# app/jobs/guests_cleanup.rb
class GuestsCleanupJob < ActiveJob::Base
  queue_as :low_priority
  #....
end

# Now your job will run on queue production.low_priority on your
# production environment and on staging.low_priority
# on your staging environment
```

If you want more control on what queue a job will be run you can pass a `:queue` option to `#set`:

```
MyJob.set(queue: :another_queue).perform_later(record)
```

To control the queue from the job level you can pass a block to `#queue_as`. The block will be executed in the job context (so you can access `self.arguments`) and you must return the queue name:

```
class ProcessVideoJob < ActiveJob::Base
  queue_as do
    video = self.arguments.first
    if video.owner.premium?
      :premium_videojobs
    else
      :videojobs
    end
  end

  def perform(video)
    # Do process video
  end
end

ProcessVideoJob.perform_later(Video.last)
```

Make sure your queueing backend "listens" on your queue name. For some backends you need to specify the queues to listen to.

6 Callbacks

Active Job provides hooks during the life cycle of a job. Callbacks allow you to trigger logic during the life cycle of a job.

6.1 Available callbacks

- `before_enqueue`
- `around_enqueue`
- `after_enqueue`
- `before_perform`
- `around_perform`
- `after_perform`

6.2 Usage